

CSS Setup and Syntax

Intro to CSS

While HTML is the fundamental structure of every web page, it can be visually unappealing on its own. Cascading Style Sheets or CSS is a language web developers use to style the HTML content on a web page. If you're interested in modifying colors, font types, font sizes, images, element positioning, and more, CSS is the tool for the job!

In this lesson, you'll learn how to set up your CSS file structure and select which HTML elements you wish to style.

Instructions

1. Take a look at the code in `index.html` and observe how it displays in the browser to the right. This is plain HTML without any styling. Let's take a quick look at the power of CSS. Copy and paste the following line of code onto line 5:

```
<link href="style.css" rel="stylesheet">
```

What happens when you press the Run button? Take some time to explore and experiment with the code in the `style.css` file.

CSS Anatomy

The diagram on the right shows two different methods, or *syntaxes*, for writing CSS code. The first syntax shows CSS applied as a *ruleset*, while the second shows it written as an *inline style*. Two different methods of writing CSS may seem a bit intimidating at first, but it's not as bad as it looks!

Both methods contain common features in their [anatomy](#). Notice how both syntaxes contain a *declaration*. Declarations are the core of CSS. They apply a style to the selected element. Here, the `<p>` element has been selected in both syntaxes and will be styled to display the text in blue.

Understanding that a declaration is used to style a selected element is key to learning how to style HTML documents with CSS! The terms below explain each of the labels in the diagram on the right.

Ruleset Terms:

Selector—The beginning of the ruleset used to target the element that will be styled.

Declaration Block—The code in-between (and including) the curly braces ({ }) that contains the CSS declaration(s).

Declaration—The group name for a property and value pair that applies a style to the selected element.

Property—The first part of the declaration that signifies what visual characteristic of the element is to be modified.

Value—The second part of the declaration that signifies the value of the property.

Inline Style Terms:

Opening Tag—The start of an [HTML element](#). This is the element that will be styled.

Attribute—The style [attribute](#) is used to add CSS inline styles to an HTML element.

Declaration—The group name for a property and value pair that applies a style to the selected element.

Property—The first part of the declaration that signifies what visual characteristic of the element is to be modified.

Value—The second part of the declaration that signifies the value of the property.

Don't worry about memorizing all of these—you will get acquainted with them more and more as the course progresses! Feel free to come back and use this exercise as a reference later on.

Instructions

Study the diagrams to become familiar with the CSS syntax and the new terms that will be used throughout the course.

Click the “Next” button when you are ready to write some code!

Inline Styles

Although CSS is a different language than HTML, it's possible to write CSS code directly within HTML code using *inline styles*.

To style an HTML element, you can add the `style` attribute directly to the opening tag. After you add the attribute, you can set it equal to the CSS style(s) you'd like applied to that element.

```
<p style='color: red;'>I'm learning to code!</p>
```

The code in the example above demonstrates how to use inline styling. The paragraph element has a `style` attribute within its opening tag. Next, the `style` attribute is set equal to `color: red;`, which will set the color of the paragraph text to red within the browser.

If you'd like to add *more* than one style with inline styles, simply keep adding to the `style` attribute. Make sure to end the styles with a semicolon (;).

```
<p style='color: red; font-size: 20px;'>I'm learning to  
code!</p>
```

It's important to know that inline styles are a quick way of directly styling an HTML element, but are rarely used when creating websites. But you may encounter circumstances where inline styling is necessary, so understanding how it works, and recognizing it in HTML code is good knowledge to have. Soon you'll learn the proper way to add CSS code!

Instructions

1. In `index.html`, use inline styling to set the `color` of the first paragraph (the first `<p>` element) to `green`.

Internal Stylesheet

As previously stated, inline styles are not the best way to style HTML elements. If you wanted to style, for example, multiple `<h1>` [elements](#), you would have to add inline styling to each element manually. In addition, you would also have to maintain the HTML code when additional `<h1>` elements are added.

Fortunately, HTML allows you to write CSS code in its own dedicated section with a `<style>` element nested inside of the `<head>` element. The CSS code inside the `<style>` element is often referred to as an *internal stylesheet*.

An internal stylesheet has certain benefits and use cases over inline styles, but once again, it's not best practice (we'll get there, we promise). Understanding how to use internal stylesheets is nonetheless helpful knowledge to have.

To create an internal stylesheet, a `<style>` element must be placed inside of the `<head>` element.

```
<head>

  <style>
```

```
</style>

</head>
```

Copy to Clipboard

After adding opening and closing `<style>` tags in the head section, you can begin writing CSS code.

```
<head>

  <style>

    p {

      color: red;

      font-size: 20px;

    }

  </style>

</head>
```

Copy to Clipboard

The CSS code in the example above changes the color of all paragraph text to red and also changes the size of the text to 20 pixels. Note how the syntax of the CSS code matches (for the most part) the syntax you used for inline styling. The main difference is that you can specify which elements to apply the styling.

Instructions

1. Let's move the inline style that was added to the paragraph into an internal stylesheet. Start by adding an empty `<style>` element in the head of `index.html`.
2. Inside of the `<style>` element, add a CSS ruleset targeting the paragraph (the `<p>` element). You can leave the declaration block empty for now.
3. Next, place just the declaration from the inline style into the empty declaration block in the inline stylesheet.
4. Finally, delete the inline style from the `<p>` element. Notice how the styling works the same in the stylesheet as it did in the inline style!

```
<!DOCTYPE html>

<html>

<head>

  <style>

    p{

      color: green;

    }

  </style>

  <title>Vacation World</title>

</head>

<body>

  <img

src='https://content.codecademy.com/courses/freelance-1/unit-2/explore.jpg' />

  <h1 class='title'>Top Vacation Spots</h1>

  <h5>By: Stacy Gray</h5>

  <h6>Published: 2 Days Ago</h6>
```

```
<p>The world is full of fascinating places. Planning the perfect vacation involves packing up, leaving home, and experiencing something new.</p>
```

```
<h2 class='destination'>1. Florence, Italy</h2>
```

```
<div class='description'>A city-size shrine to the Renaissance, Florence offers frescoes, sculptures, churches, palaces, and other monuments from the richest cultural flowering the world has known. Names from its dazzling historical past; Dante, Michelangelo, Galileo, Machiavelli are some of the most resonant of the medieval age. <a href='https://www.nationalgeographic.com/travel/destination/florence' target='_blank'>Learn More</a>.
```

```
<h5>Top Attractions</h5>
```

```
<ul>
```

```
<li>Museums</li>
```

```
<li>Bike Tours</li>
```

```
<li>Historical Monuments</li>
```

```
</ul>
```

```
</div>
```

```
<h2 class='destination'>2. Beijing, China</h2>
```

```
<div class='description'>A city in the midst of reinventing itself and continuing to build on the success of the 2008 Summer Olympics, Beijing is a place of frenzied construction. New housing, new roads, and new sports venues seem to spring up overnight. At the same time, the capital of the Peoples Republic of China remains an epicenter of tradition, with the treasures of nearly 2,000 years as the imperial
```

capital still on view in the famed Forbidden City and in the luxuriant pavilions and gardens of the Summer Palace.

[Learn More](https://www.nationalgeographic.com/travel/destination/beijing).

Top Attractions

- Biking

- Historical Sites

- Restaurants and Dining

3. Seoul, South Korea

The Korean capital is a city of contrasts. Fourteenth-century city gates squat in the shadow of 21st-century skyscrapers, while the broad Han River is back-dropped by granite mountains rising in the city center complete with alpine highways speeding around their contours and temples nestling among their crags. Fashionable, gadget-laden youths battle for sidewalk space with fortune-tellers and peddlers, while tiny neighborhoods of traditional cottages contrast with endless ranks of identical apartments.

[Learn More](https://www.nationalgeographic.com/travel/destination/seoul).

Top Attractions


```

        <li>Parasailing</li>

        <li>Segway Tours</li>

        <li>Spas and Resorts</li>

    </ul>

</div>


<h2> More Destinations </h2>

<ul>

    <li><h4 class='destination'>Jackson Hole, Wyoming</h4></li>

    <li><h4 class='destination'>Cape Town, South Africa</h4></li>

    <li><h4 class='destination'>La Paz, Bolivia</h4></li>

</ul>


<p>&mdash;Best of luck with your travels, and be sure to send
pictures and stories. We'd love to hear them!</p>

</body>

</html>

```

External Stylesheet

Developers avoid mixing code by storing HTML and CSS code in separate files (HTML files contain only HTML code, and CSS files contain only CSS code).

You can create an external stylesheet by using the `.css` file name extension, like so:
`style.css`

With an external stylesheet, you can write all the CSS code needed to style a page without sacrificing the readability and maintainability of your HTML file.

Instructions

1. Take a look at `index.html`. Cut the CSS code ruleset in between the opening and closing `<style>` tags and paste it directly into the file called `style.css`. Delete the remaining `<style>` element (now empty) from `index.html` and press the Run button. Hmm, the font changes didn't take effect? Click the Next button to find out why.

```
p {  
  
    color: green;  
  
}
```

Linking the CSS File

Perfect! We successfully separated structure (HTML) from styling (CSS), but the web page still looks bland. Why?

When HTML and CSS codes are in separate files, the files must be linked. Otherwise, the HTML file won't be able to locate the CSS code, and the styling will not be applied.

You can use the `<link>` element to link HTML and CSS files together. The `<link>` element must be placed within the head of the HTML file. It is a self-closing tag and requires the following attributes:

`href` — like the anchor element, the value of this attribute must be the address, or path, to the CSS file.

`rel` — this attribute describes the relationship between the HTML file and the CSS file. Because you are linking to a stylesheet, the value should be set to `stylesheet`.

When linking an HTML file and a CSS file together, the `<link>` element will look like the following:

```
<link href='https://www.codecademy.com/stylesheets/style.css'  
rel='stylesheet'>
```

Note that in the example above, the path to the stylesheet is a URL:

```
https://www.codecademy.com/stylesheets/style.css
```

Specifying the path to the stylesheet using a URL is one way of linking a stylesheet.

If the CSS file is stored in the same [directory](#) as your HTML file, then you can specify a [relative path](#) instead of a URL, like so:

```
<link href='./style.css' rel='stylesheet'>
```

Using a relative path is very common way of linking a stylesheet.

Instructions

1. Let's link the stylesheet `style.css` to the HTML file `index.html`. First, add a `<link>` element within the `<head>` section.
2. Next, add the `href` attribute to the `<link>` element and set it equal to `style.css`. Take a look at the web page in the browser to the right. Do you notice any changes yet?
3. Finally, add the `rel` attribute and set it to the correct value. Keep an eye on the first paragraph's font—it should appear different from the destinations' descriptions when your stylesheet is linked correctly.

```
<head>  
  
  <link href="style.css" rel='stylesheet'>  
  
  <title>Vacation World</title>  
  
</head>
```

CSS Selectors

Type

Remember that *declarations* are a fundamental part of CSS because they apply a style to a selected element. But how do you decide which elements will get the style? With a [selector](#).

A selector is used to target the specific HTML element(s) to be styled by the declaration. One selector you may already be familiar with is the *type* selector. Just like its name suggests, the type selector matches the *type* of the element in the HTML document.

In the previous lesson, you changed the color of a paragraph element.

```
p {  
  
    color: green;  
  
}
```

Copy to Clipboard

This is an instance of using the type selector! The element type is `p`, which comes from the HTML `<p>` element.

Some important notes on the type selector:

- The type selector does not include the angle brackets.

- Since element types are often referred to by their opening tag name, the type selector is sometimes referred to as the *tag name* or *element* selector.

Instructions

1. Open `style.css`. On line 5, add a ruleset using the type selector to target all `<h1>` elements. Leave the declaration block (`{ }`) empty for now.

2. Inside the curly braces of the `h1` selector, add the declaration below:

```
color: maroon;
```

Note that the content of the web page will update because we've already linked

`style.css` to `index.html`.

Universal

You learned how the *type selector* selects all elements of a given type. Well, the *universal selector* selects all elements of *any* type.

Targeting all of the elements on the page has a few specific use cases, such as resetting default browser styling or selecting all children of a parent element. Don't worry if you don't understand the use cases right now; we will get to them later on in our Learn CSS journey.

The universal selector uses the `*` character in the same place where you specified the type selector in a ruleset, like so:

```
* {  
  
    font-family: Verdana;  
  
}
```

In the code above, every text element on the page will have its font changed to `Verdana`.

Instructions

1. To see how the universal selector targets all elements on a page, copy the rule below and paste it on the first line of **`style.css`**.

```
* {  
  
    border: 1px solid red;  
  
}
```

Then, click “Run”. Since the universal selector targets all elements, every element on the page now has a red border. Not a visually pleasing look, but you can see how all of the elements have been modified.

Class

CSS is not limited to selecting elements by their type. As you know, HTML elements can also have attributes. When working with HTML and CSS a [class](#) attribute is one of the most common ways to select an element.

For example, consider the following HTML:

```
<p class='brand'>Sole Shoe Company</p>
```

The paragraph element in the example above has a `class` attribute within the opening tag of the `<p>` element. The `class` attribute is set to `'brand'`. To select this element using CSS, we can create a ruleset with a class selector of `.brand`.

```
.brand {
```

To select an HTML element by its class using CSS, a period (.) must be prepended to the class's name. In the example above, the class is `brand`, so the CSS selector for it is `.brand`.

Instructions

1. On line 11 of **index.html** there is an `<h1>` element with the class `title`. Now, open **style.css**. On line 13, use the class selector to create a ruleset that selects that HTML element using the class `title`.

2. Inside the curly braces of the `.title` selector, add the declaration:

```
color: teal;
```

This code will change the color of the `h1` with the `title` class to teal. We'll see in a later exercise why using `.title` overrides the `h1` selector.

Multiple Classes

We can use CSS to select an HTML element's `class` attribute by name. And so far, we've selected elements using only one class name per element. If every HTML element had a single class, all the style information for each element would require a new class.

Luckily, it's possible to add more than one class name to an HTML element's `class` attribute.

For instance, perhaps there's a heading element that needs to be green and bold. You could write two CSS rulesets like so:

```
.green {  
  
    color: green;  
  
}  
  
.bold {  
  
    font-weight: bold;  
}
```

Then, you could include both of these classes on one HTML element like this:

```
<h1 class='green bold'> ... </h1>
```

We can add multiple classes to an HTML element's `class` attribute by separating them with a space. This enables us to mix and match CSS classes to create many unique styles without writing a custom class for every style combination needed.

Instructions

1. In **index.html**, on line 11, there is an `<h1>` element with a class of `title`. Add a second class named `uppercase` to this element.

2. Next, navigate to **style.css**, and add a ruleset using the `.uppercase` class selector. Then, add the declaration below inside of the curly braces.

```
text-transform: uppercase;
```

ID

Oftentimes it's important to select a single element with CSS to give it its own unique style. If an HTML element needs to be styled uniquely, we can give it an ID using the `id` attribute.

```
<h1 id='large-title'> ... </h1>
```

In contrast to `class` which accepts multiple values, and can be used broadly throughout an HTML document, an element's `id` can only have a single value, and only be used once per page.

To select an element's ID with CSS, we prepend the `id` name with a number sign (`#`). For instance, if we wanted to select the HTML element in the example above, it would look like this:

```
#large-title {}
```

The `id` name is `large-title`, therefore the CSS selector for it is `#large-title`.

Instructions

1. On line 11 of **index.html**, inside the `h1` opening tag and after the class attribute, add an `id` with the value `article-title`.

2. Navigate to **style.css**. Add a ruleset using the ID selector to target the `article-title` ID. Inside of its curly braces, write the declaration:

```
font-family: cursive;
```

You'll see the title change to a cursive font bringing some beauty and elegance to the page! Okay, maybe not so much. But the font does change.

Attribute

You may remember that some HTML elements use attributes to add extra detail or functionality to the element. Some familiar attributes may be `href` and `src`, but there are [many more](#)—including `class` and `id`!

The *attribute selector* can be used to target HTML elements that already contain attributes. Elements of the same type can be targeted differently by their attribute or attribute value. This alleviates the need to add new code, like the `class` or `id` attributes.

Attributes can be selected similarly to types, classes, and IDs.

```
[href] {
```

```
color: magenta;

}
```

The most basic syntax is an attribute surrounded by square brackets. In the above example: `[href]` would target all elements with an `href` attribute and set the `color` to `magenta`.

And it can get [more granular](#) from there by adding type and/or attribute values. One way is by using `type[attribute*=value]`. In short, this code selects an element where the attribute contains any instance of the specified value. Let's take a look at an example.

```
<img src='/images/seasons/cold/winter.jpg'>

<img src='/images/seasons/warm/summer.jpg'>
```

The HTML code above renders two `<img>` elements, each containing a `src` attribute with a value equaling a link to an image file.

```
img[src*='winter'] {

    height: 50px;

}

img[src*='summer'] {

    height: 100px;

}
```

Now take a look at the above CSS code. The *attribute selector* is used to target each image individually.

The first ruleset looks for an `img` element with an attribute of `src` that contains the string `'winter'`, and sets the `height` to `50px`.

The second ruleset looks for an `img` element with an attribute of `src` that contains the string `'summer'`, and sets the `height` to `100px`.

Notice how no new HTML markup (like a class or id) needed to be added, and we were still able to modify the styles of each image independently. This is one advantage to using the attribute selector!

Instructions

1.To use the attribute selector to select the `<a>` element with an `href` attribute value containing `'florence'`, add the following code to **style.css**:

```
a[href*='florence'] {  
  
  color: lightgreen;  
  
}
```

Notice how only the “Learn More” link in the Florence, Italy paragraph changed to light green.

2.Next, in **style.css**, use the attribute selector to select the `<a>` element that has an `href` attribute value containing `'beijing'`. Add a declaration of `color: lightblue;` to make the link appear light blue.

3.Finally, use the attribute selector change the `<a>` element that has an `href` attribute value containing `'seoul'` to `lightpink`.

Pseudo-class

You may have observed how the appearance of certain elements can change, or be in a different state, after certain user interactions. For instance:

When you click on an `<input>` element, and a blue border is added showing that it is in *focus*. When you click on a blue `<a>` link to *visit* to another page, but when you return the link's text is purple. When you're filling out a form and the submit button is grayed out and *disabled*. But when all of the fields have been filled out, the button has color showing that it's *active*.

These are all examples of pseudo-class selectors in action! In fact, `:focus`, `:visited`, `:disabled`, and `:active` are all pseudo-classes. Factors such as user interaction, site navigation, and position in the document tree can all give elements a different state with pseudo-class.

A pseudo-class can be attached to any selector. It is always written as a colon `:` followed by a name. For example `p:hover`.

```
p:hover {  
  
    background-color: lime;  
  
}
```

In the above code, whenever the mouse hovers over a paragraph element, that paragraph will have a lime-colored background.

Instructions

1. Add a ruleset to the end of **style.css** that selects the `<a>` elements on the page. Leave the declaration block empty for now.
2. Next, add a `:hover` pseudo-class to the `a` selector you just created.
3. Lastly, set the text color to dark orange by adding the following CSS declaration inside the declaration block:

```
color: darkorange;
```

Now when you hover the mouse over the “Learn More” text, the font color changes to dark orange! Neato.

Classes and IDs

CSS can select HTML elements by their type, class, and ID. CSS classes and IDs have different purposes, which can affect which one you use to style HTML elements.

CSS classes are meant to be reused over many elements. By writing CSS classes, you can style elements in a variety of ways by mixing classes. For instance, imagine a page with two headlines. One headline needs to be bold and blue, and the other needs to be bold and green. Instead of writing separate CSS rules for each headline that repeat each other’s code, it’s better to write a `.bold` CSS rule, a `.green` CSS rule, and a `.blue` CSS rule. Then you can give one headline the `bold green` classes, and the other the `bold blue` classes.

While classes are meant to be used many times, an ID is meant to style only one element. As you’ll learn in the next exercise, IDs override the styles of types and classes. Since IDs override these styles, they should be used sparingly and only on elements that need to always appear the same.

Instructions

1.To demonstrate using classes on multiple elements, let’s give a few elements the same style.In **index.html**, there are four `<h2>` elements. Give each of them a class of `heading-background`.

2.Now, let’s give a unique style to a single element using ID.On line 13 of **index.html**, there’s an `<h6>` element that displays the time the article on the page was published.Add an id attribute to the `<h6>`, and give it a value of `publish-time`.

3.Then, in **style.css**, create a ruleset targeting the new `heading-background` class, and give it a declaration of:

```
background-color: aqua;
```

After you press Run, notice how all of the `<h2>` elements now have the same style. This `heading-background` class can continue to be applied to any element you wish to bestow that amazing style onto.

4. Finally, in **style.css**, create another ruleset using the `publish-time` ID selector. Add the declaration:

```
color: gray;
```

Since ID's are single-use, this element now has a unique ID that can't be used again in this document!

Specificity

Specificity is the order by which the browser decides which CSS styles will be displayed. A best practice in CSS is to style elements while using the lowest degree of specificity so that if an element needs a new style, it is easy to override.

IDs are the most specific selector in CSS, followed by classes, and finally, type. For example, consider the following HTML and CSS:

```
<h1 class='headline'>Breaking News</h1>

h1 {

    color: red;

}

.headline {

    color: firebrick;

}
```

In the example code above, the color of the heading would be set to `firebrick`, as the class selector is more specific than the type selector. If an ID attribute (and selector) were added to the code above, the styles within the ID selector's body would override all other styles for the heading. Over time, as files grow with code, many elements may have IDs, which can make CSS difficult to edit since a new, more specific style must be created to change the style of an element. To make styles easy to edit, it's best to style with a type selector, if possible. If not, add a class selector. If that is not specific enough, then consider using an ID selector.

Instructions

1. To compare type, class, and ID specificity, let's add a class and ID to an existing element. On line 12 of `index.html`, there is an `<h5>` element with the author's name. Give the element a class of `author-class` and an ID of `author-id`.

2. The element now has 3 ways of selecting it, by type, class, and ID. Add the 3 following rulesets to the bottom of `style.css` utilizing each of the selectors:

```
h5 {  
    color: yellow;  
}  
  
.author-class {  
    color: pink;  
}  
  
#author-id {  
    color: cornflowerblue;  
}
```

Each of these rules selects the same element in a different way. Which style will win the "specificity war"? Click "Run" to find out! Because ID is the most specific selector, the element will change to cornflower blue. You may have noticed the other `<h5>` elements

changed to yellow. This is because the most specific (and only) selector they have is their type.

Chaining

When writing CSS rules, it's possible to require an HTML element to have two or more CSS selectors at the same time.

This is done by combining multiple selectors, which we will refer to as chaining. For instance, if there was a `special` class for `<h1>` elements, the CSS would look like below:

```
h1.special { }
```

The code above would select only the `<h1>` elements with a class of `special`. If a `<p>` element also had a class of `special`, the rule in the example would not style the paragraph.

Instructions

1. Let's use chaining to select only the `<h2>` elements with destinations, and add a style to them. In **style.css**, write a CSS selector for `<h2>` elements with a class of `.destination`. Inside the selector's curly braces, add this declaration:

```
font-family: Tahoma;
```

This will change the font of the `h2` elements that also have the class `destination`. The last `<h2>` element ("More Destinations") will remain unchanged.

Descendant Combinator

In addition to chaining selectors to select elements, CSS also supports selecting elements that are nested within other HTML elements, also known as *descendants*. For instance, consider the following HTML:


```
<ul class='main-list'>

  <li> ... </li>

  <li> ... </li>

  <li> ... </li>

</ul>
```

The nested `<li>` elements are descendants of the `<ul>` element and can be selected with the *descendant combinator* like so:

```
.main-list li {}
```

In the example above, `.main-list` selects the element with the `.main-list` class (the `<ul>` element). The descendant `<li>`'s are selected by adding `li` to the selector, separated by a space. This results in `.main-list li` as the final selector.

Selecting elements in this way can make our selectors even more specific by making sure they appear in the context we expect.

Instructions

1. In `index.html`, each destination has a paragraph with a `description` class below it. Nested within each description paragraph, is an `<h5>` element with the text "Top Attractions". They're a little hard to read since they turned yellow. Let's fix that! Navigate to `style.css`. Add a ruleset that uses the descendant combinator to target only the `<h5>` descendants of elements with the class `.description`.

2. Inside the curly braces of the selector, add a declaration of:

```
color: blueviolet;
```

Chaining and Specificity

In the last exercise, instead of selecting all `<h5>` elements, you selected only the `<h5>` elements nested inside the `.description` elements. This CSS selector was more specific than writing only `h5`. Adding more than one tag, class, or ID to a CSS selector increases the specificity of the CSS selector.

For instance, consider the following CSS:

```
p {  
  
    color: blue;  
  
}  
  
.main p {  
  
    color: red;  
  
}
```

Both of these CSS rules define what a `<p>` element should look like. Since `.main p` has a class and a `p` type as its selector, only the `<p>` elements inside the `.main` element will appear `red`. This occurs despite there being another more general rule that states `<p>` elements should be `blue`.

Instructions

1. To show how specificity increases with additional selectors, let's create another ruleset with the descendant combinator and then compare it to a ruleset without. In

style.css, write a ruleset using the descendant combinator to select the `<h4>` elements nested in the `<li>` elements. Inside of the curly braces, write:

```
color: gold;
```

Click Run and then scroll down the page to see the `<h4>` elements under “More Destinations” appear gold.

2. Now, create a ruleset targeting elements with just the `h4` type, and add a declaration of:

```
color: dodgerblue;
```

Will the `<h4>` elements turn blue? Click “Run” to find out. The elements stay gold because there is a more specific selector for `<h4>` elements you wrote in the last step. Because of the more specific CSS selector (`li h4`), the more general selector of `h4` will not take hold.

Multiple Selectors

In order to make CSS more concise, it’s possible to add CSS styles to multiple CSS selectors all at once. This prevents writing repetitive code.

For instance, the following code has repetitive style attributes:

```
h1 {  
  
    font-family: Georgia;  
  
}
```

```
.menu {  
  
  font-family: Georgia;  
  
}
```

Instead of writing `font-family: Georgia` twice for two selectors, we can separate the selectors by a comma to apply the same style to both, like this:

```
h1,  
  
.menu {  
  
  font-family: Georgia;  
  
}
```

By separating the CSS selectors with a comma, both the `<h1>` elements and the elements with the `menu` class will receive the `font-family: Georgia` styling.

Instructions

1. In **style.css**, write selectors for the `<h5>` and `<li>` elements so they will both be styled with the same CSS rule. Apply this style declaration:

```
font-family: monospace;
```

Notice that both the `<h5>` and `<li>` element's fonts will change, without writing the same CSS declaration twice.

```
* {  
  
    border: 1px solid red;  
  
}  
  
p {  
  
    color: green;  
  
}  
  
h1 {  
  
    color: maroon;  
  
}  
  
.title {  
  
    color: teal;  
  
}  
  
.uppercase {  
  
    text-transform: uppercase;  
  
}
```

```
#article-title {

    font-family: Tahoma;

}

a[href*='florence'] {

    color: lightgreen;

}

a[href*='beijing'] {

    color: lightblue;

}

a[href*='seoul'] {

    color: lightpink;

}

a:hover {

    color:darkorange;

}
```

```
.heading-background {  
  
    background-color: aqua;  
  
}
```

```
#publish-time {  
  
    color: lightgray;  
  
}
```

```
h5 {  
  
    color: yellow;  
  
}
```

```
.author-class {  
  
    color: pink;  
  
}
```

```
#author-id {  
  
    color: cornflowerblue;  
  
}
```

```
h2.destination {  
  
    font-family: Tahoma;  
  
}
```

```
.description h5 {  
  
    color: blueviolet;  
  
}
```

```
li h4 {  
  
    color: gold;  
  
}
```

```
h4 {  
  
    color: dodgerblue;  
  
}
```

```
h5,li {  
  
    font-family: monospace;  
  
}
```


CSS Visual Rules

Introduction To Visual Rules

The purpose of CSS is to add style to web page, and each element on the page can have many style properties. Some of the basic properties relate to the size, style, and color of the element. In this lesson, you'll learn some fundamental CSS visual rules that you can use to start styling web page elements!

Instructions

Explore the code to the right. Think about how it relates to the web page on the right side of the browser.

Font Family

If you've ever used word processing software, like Microsoft Word or Google Docs, chances are that you probably also used a feature that allowed you to change the font you were typing in. Font refers to the technical term [typeface](#), or *font family*.

To change the typeface of text on your web page, you can use the `font-family` property.

```
h1 {  
  
    font-family: Garamond;  
  
}
```

In the example above, the font family for all main heading elements has been set to `Garamond`.

When setting typefaces on a web page, keep the following points in mind:

The font specified must be installed on the user's computer or downloaded with the site.

[Web safe fonts](#) are a group of fonts supported across most browsers and operating systems. Unless you are using web safe fonts, the font you choose may not appear the same between all browsers and operating systems. When the name of a typeface consists of more than one word, it's a best practice to enclose the typeface's name in quotes, like so:

```
h1 {  
  
    font-family: 'Courier New';  
  
}
```

You'll take a deeper look into typography in a later lesson!

Instructions

1. Inside **style.css**, set the font family of the main heading (`h1`) and subheading (`h2`) to `Georgia`. Note that certain font changes may be hard to recognize at first. Feel free to remove and add the declaration a few times until you notice the change!

2. Next, add a style rule that sets the font family of the paragraph (`p`) to `Helvetica`.

Font Size

Changing the typeface isn't the only way to customize the text. Oftentimes, different sections of a web page are highlighted by modifying the *font size*.

To change the size of text on your web page, you can use the `font-size` property.

```
p {  
  
    font-size: 18px;  
  
}
```

In the example above, the `font-size` of all paragraphs was set to `18px`. `px` means pixels, which is one way to measure font size.

Instructions

1. In **style.css**, set the `font-size` of paragraph (`p`) elements to 18 pixels.

Font Weight

In CSS, the `font-weight` property controls how bold or thin text appears.

```
p {  
  
    font-weight: bold;  
  
}
```

In the example above, all paragraphs on the web page would appear bolded.

The `font-weight` property has another value: `normal`. Why does it exist?

If we wanted *all* text on a web page to appear bolded, we could select all text elements and change their font weight to `bold`. If a certain section of text was required to appear normal, however, we could set the font weight of that particular element to `normal`, essentially shutting off bold for that element.

Instructions

1. In **style.css**, set the font weight of paragraph (`p`) elements to `bold`.

Text Align

No matter how much styling is applied to text (typeface, size, weight, etc.), the text always appears on the left side of the container in which it resides.

To align text we can use the `text-align` property. The `text-align` property will align text to the element that holds it, otherwise known as its *parent*.

```
h1 {
```

```
text-align: right;

}
```

The `text-align` property can be set to one of the following commonly used values:

`left` — aligns text to the left side of its parent element, which in this case is the browser.

`center` — centers text inside of its parent element.

`right` — aligns text to the right side of its parent element.

`justify` — spaces out text in order to align with the right and left side of the parent element.

Instructions

1. In **style.css**, set the `text-align` property of the main heading (`h1`) so that it appears in the center.

Color and Background Color

Before discussing the specifics of color, it's important to make two distinctions about color. Color can affect the following design aspects: Foreground color, Background color

Foreground color is the color that an element appears in. For example, when a heading is styled to appear green, the *foreground color* of the heading has been styled.

Conversely, when a heading is styled so that its background appears yellow, the *background color* of the heading has been styled.

In CSS, these two design aspects can be styled with the following two properties:

`color`: this property styles an element's foreground color

`background-color`: this property styles an element's background color

```
h1 {  
  
    color: red;  
  
    background-color: blue;  
  
}
```

In the example above, the text of the heading will appear in red, and the background of the heading will appear blue.

Instructions

1. Take a look at the caption on the image at the bottom of the page. In **style.css**, set the background color in the `.caption` selector to `white`.
2. Then, in the same ruleset, set the color of the text to `black`. That should make the text a bit easier to read!

Opacity

Opacity is the measure of how transparent an element is. It's measured from 0 to 1, with 1 representing 100%, or fully visible and opaque, and 0 representing 0%, or fully invisible.

Opacity can be used to make elements fade into others for a nice overlay effect. To adjust the opacity of an element, the syntax looks like this:

```
.overlay {  
  
    opacity: 0.5;  
  
}
```

In the example above, the `.overlay` element would be 50% visible, letting whatever is positioned behind it show through.

Instructions

1. Let's give the caption on the image some transparency! In **style.css**, set `.caption` to have `0.75` opacity.

Background Image

CSS has the ability to change the background of an element. One option is to make the background of an element an image. This is done through the CSS property

`background-image`. Its syntax looks like this:

```
.main-banner {  
  
  background-image: url('https://www.example.com/image.jpg');  
  
}
```

Copy to Clipboard

The `background-image` property will set the element's background to display an image.

The value provided to `background-image` is a `url`. The `url` should be a URL to an image. The `url` can be a file within your project, or it can be a link to an external site. To link to an image inside an existing project, you must provide a [relative file path](#). If there was an image folder in the project, with an image named

`mountains.jpg`, the relative file path would look like below:

```
.main-banner {  
  
  background-image: url('images/mountains.jpg');  
  
}
```

Instructions

1. In **style.css**, change the background image of the element with the `.image` class. The image is stored in the following URL:

<https://content.codecademy.com/courses/freelance-1/unit-2/soccer.jpeg>

Important

`!important` can be applied to specific declarations, instead of full rules. It will override *any* style no matter how specific it is. As a result, it should almost never be used. Once `!important` is used, it is very hard to override.

The syntax of `!important` in CSS looks like this:

```
p {  
  
  color: blue !important;  
  
}  
  
.main p {  
  
  color: red;  
  
}
```

Since `!important` is used on the `p` selector's `color` attribute, all `p` elements will appear blue, even though there is a more specific `.main p` selector that sets the `color` attribute to red.

One justification for using `!important` is when working with multiple stylesheets. For example, if we are using the [Bootstrap](#) CSS framework and want to override the styles for one specific HTML element, we can use the `!important` property.

Instructions

1. Imagine `style-library.css` is a stylesheet that is full of good styles! But, you don't like how it is turning the color of `<h1>` yellow. In `style.css`, add an `!important` rule to the color style of inside the `h1` ruleset to change the color back to `#FFF` (white).

```
body {  
  
    /* Old browsers */  
  
    background: #141E30;  
  
    /* Chrome 10-25, Safari 5.1-6 */  
  
    background: -webkit-linear-gradient(-45deg, #35577D, #141E30);  
  
    /* W3C, IE 10+/ Edge, Firefox 16+, Chrome 26+, Opera 12+, Safari 7+ */  
  
    background: linear-gradient(-45deg, #35577D, #141E30);  
  
    margin: 0;  
  
    padding: 0;  
  
}  
  
h1 {  
  
    color: #FFF !important;  
  
    font-size: 2em;  
  
    padding-top: 100px;  
  
    width: 100%;  
  
    font-family: Georgia;  
  
    text-align: center;  
  
}
```



```
h2 {

    border-bottom: 1px solid rgba(255, 255, 255, 0.5);

    color: rgba(255, 255, 255, 0.5);

    font-weight: 100;

    font-size: 22px;

    line-height: 24px;

    padding-bottom: 30px;

    text-align: left;

    width: 70%;

    font-family: Georgia;

}

p {

    color: aliceblue;

    line-height: 1.3em;

    text-align: left;

    width: 100%;

    font-family: Helvetica;

    font-size: 18px;

    font-weight: bold;

}
```

```
.byline {

    font-family: Helvetica;

    color: rgba(255, 255, 255, 0.5);

    float: left;

    font-size: 14px;

    padding-left: 10px;

    text-transform: uppercase;
}

.caption {

    display: block;

    font-family: 'Playfair Display', serif;

    font-size: 14px;

    font-style: italic;

    line-height: 14px;

    margin-left: 20px;

    padding: 10px;

    position: relative;

    top: 80%;

    width: 60%;

    background-color: white;
```

```
    color: black;

}

.content {

    padding: 40px;

}

.image {

    background-image:
url('https://content.codecademy.com/courses/freelance-1/unit-2/soccer.jpeg
');

    background-size: cover;

    background-position: center;

    height: 300px;

}

.writer-img {

    -webkit-box-shadow: 5px 0px 5px 0px rgba(0, 0, 50, 0.97);

    -moz-box-shadow: 5px 0px 5px 0px rgba(0, 0, 50, 0.97);

    box-shadow: 5px 0px 5px 0px rgba(0, 0, 50, 0.97);

    float: left;

    width: 50px;

}
```

CSS Box Model

Introduction to the Box Model

Browsers load HTML elements with default position values. This often leads to an unexpected and unwanted user experience while limiting the views you can create. In this lesson, you will learn about the *box model*, an important concept to understand how elements are positioned and displayed on a website.

If you have used HTML and CSS, you have unknowingly seen aspects of the box model. For example, if you have set the background color of an element, you may have noticed that the color was applied not only to the area directly behind the element but also to the area to the right of the element. Also, if you have aligned text, you know it is aligned relative to something. What is that something?

All elements on a web page are interpreted by the browser as “living” inside of a box. This is what is meant by the box model.

For example, when you change the background color of an element, you change the background color of its entire box.

In this lesson, you'll learn about the following aspects of the box model:

- The dimensions of an element's box.

- The borders of an element's box.

- The paddings of an element's box.

- The margins of an element's box.

The Box Model

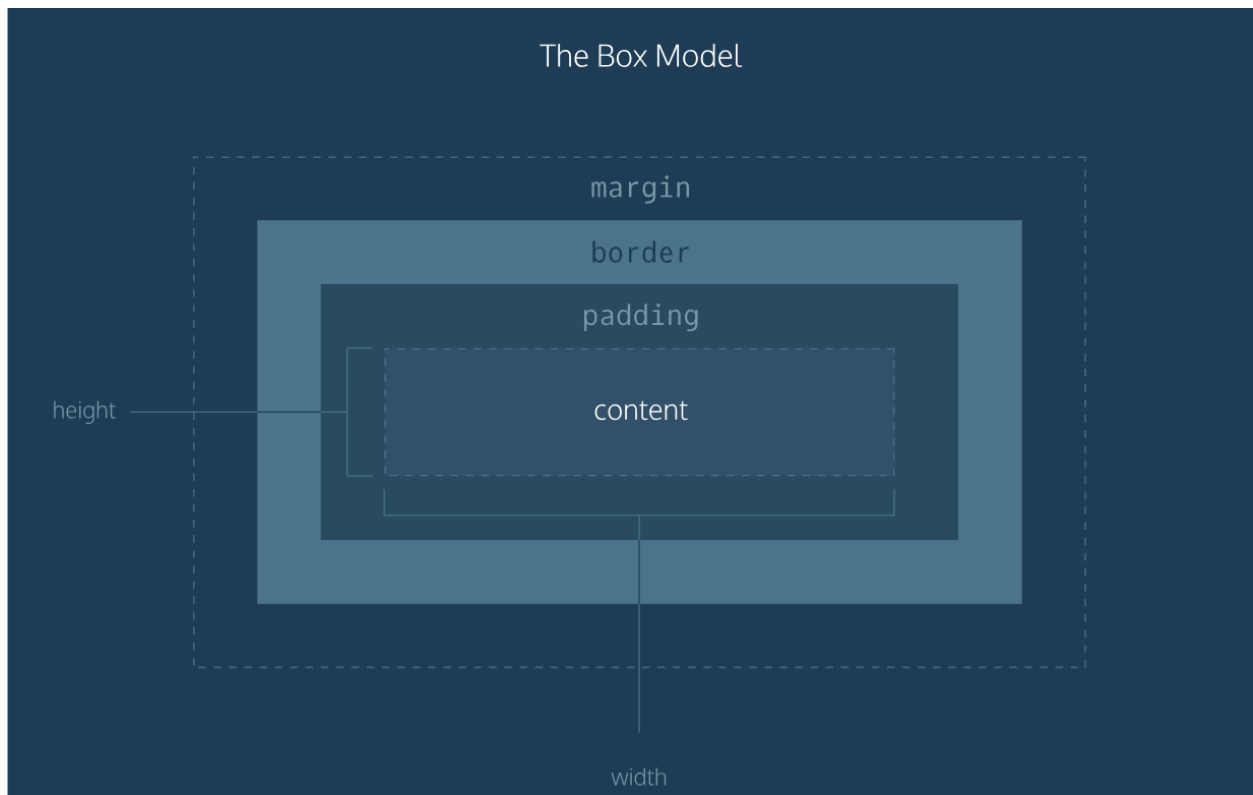
The box model comprises the set of properties that define parts of an element that take up space on a web page. The model includes the content area's size (*width* and *height*) and the element's *padding*, *border*, and *margin*. The properties include:

`width` and `height`: The width and height of the content area.

`padding`: The amount of space between the content area and the border.

`border`: The thickness and style of the border surrounding the content area and padding.

`margin`: The amount of space between the border and the outside edge of the element.



Height and Width

An element's content has two dimensions: a height and a width. By default, the dimensions of an HTML box are set to hold the raw contents of the box.

The CSS `height` and `width` properties can be used to modify these default dimensions.

```
p {  
  
  height: 80px;  
}
```

```
width: 240px;

}
```

In this example, the `height` and `width` of paragraph elements are set to 80 pixels and 240 pixels, respectively — the `px` in the code above stands for *pixels*.

Pixels allow you to set the exact size of an element's box (width and height). When the width and height of an element are set in pixels, it will be the same size on all devices — an element that fills a laptop screen will overflow a mobile screen.

Instructions

1. Add a height of 700 pixels to `#banner`.
2. Set `.pull-quote` width to 350 pixels.
3. Set the `#banner .content h1` width to 400 pixels.

Borders

A *border* is a line that surrounds an element, like a frame around a painting. Borders can be set with a specific `width`, `style`, and `color`:

`width`—The thickness of the border. A border's thickness can be set in pixels or with one of the following keywords: `thin`, `medium`, or `thick`.

`style`—The design of the border. Web browsers can render any of [10 different styles](#). Some of these styles include: `none`, `dotted`, and `solid`.

`color`—The color of the border. Web browsers can render colors using a few different formats, including [140 built-in color keywords](#).

```
p {

  border: 3px solid coral;
```

```
}
```

Copy to Clipboard

In the example above, the border has a width of 3 pixels, a style of `solid`, and a color of `coral`. All three properties are set in one line of code.

The default border is `medium none color`, where `color` is the current color of the element. If `width`, `style`, or `color` are not set in the CSS file, the web browser assigns the default value for that property.

```
p.content-header {  
  
    height: 80px;  
  
    width: 240px;  
  
    border: solid coral;  
  
}
```

In this example, the border style is set to `solid` and the color is set to `coral`. The border width is not set, so it defaults to `medium`.

Instructions

1. Add a dotted red border with 1-pixel thickness to all `h2` headings.
2. Add a solid, white border, with a 3 pixel width, to the `#banner .content h1` rule.

Border Radius

Ever since we revealed the borders of boxes, you may have noticed that the borders highlight the true shape of an element's box: square. Thanks to CSS, a border doesn't have to be square.

You can modify the corners of an element's border box with the `border-radius` property.

```
div.container {  
  
  border: 3px solid blue;  
  
  border-radius: 5px;  
  
}
```

Copy to Clipboard

The code in the example above will set *all four corners* of the border to a radius of 5 pixels (i.e. the same curvature that a circle with a radius of 5 pixels would have).

You can create a border that is a perfect circle by first creating an element with the same width and height, and then setting the radius equal to half the width of the box, which is 50%.

```
div.container {  
  
  height: 60px;  
  
  width: 60px;  
  
  border: 3px solid blue;  
  
  border-radius: 50%;  
  
}
```

The code in the example above creates a `<div>` that is a perfect circle.

Instructions

1. In **style.css**, set the border radius of `#banner .content h1` to 15 pixels. Additionally, try experimenting with other border-radius values and running your code to see the result!

Padding

The space between the contents of a box and the borders of a box is known as *padding*.

Padding is like the space between a picture and the frame surrounding it. In CSS, you can modify this space with the `padding` property.

```
p.content-header {  
  
    border: 3px solid coral;  
  
    padding: 10px;  
  
}
```

Copy to Clipboard

The code in this example puts 10 pixels of space between the content of the paragraph (the text) and the borders, on all four sides.

The `padding` property is often used to expand the background color and make the content look less cramped.

If you want to be more specific about the amount of padding on each side of a box's content, you can use the following properties:

`padding-top`

`padding-right`

`padding-bottom`

`padding-left`

Each property affects the padding on only one side of the box's content, giving you more flexibility in customization.

```
p.content-header {
```

```
border: 3px solid fuchsia;

padding-bottom: 10px;

}
```

In the example above, only the bottom side of the paragraph's content will have a padding of 10 pixels.

Instructions

1. In a single line, set the `.navigation li` elements to have 20 pixels of padding. Click "Run" and observe the changes.
2. Look at the red boxes at the bottom of the web page. Set the `.share a` elements to have 14 pixels of padding in **style.css** and run your code. Observe how the red boxes at the bottom of the page changed.
3. Set the top and bottom padding of `h2` elements to 20 pixels and set the left and right padding of `h2` elements to 30 pixels.

```
h2 {

border: 1px dotted red;

color: red;

font-size: 14px;

line-height: 48px;

text-align: center;

padding-top: 20px;

padding-bottom: 20px;

padding-right: 30px;

}
```

```
padding-left: 30px;

}
```

Padding Shorthand

Another implementation of the `padding` property lets you specify exactly how much padding there should be on each side of the content in a single declaration. A declaration that uses multiple properties as values is known as a *shorthand property*.

Padding shorthand lets you specify all of the `padding` properties as values on a single line:

```
padding-top
padding-right
padding-bottom
padding-left
```

You can specify these properties in a few different ways:

4 Values

```
p.content-header {

  padding: 6px 11px 4px 9px;

}
```

Copy to Clipboard

In the example above, the four values `6px 11px 4px 9px` correspond to the amount of padding on each side, in a clockwise rotation. In order, it specifies the padding-top value

(6px), the padding-right value (11px), the padding-bottom value (4px), and the padding-left value (9px) of the content.

3 Values

```
p.content-header {  
  
    padding: 5px 10px 20px;  
  
}
```

Copy to Clipboard

If the left and right sides of the content can be equal, the padding shorthand property allows for 3 values to be specified. The first value sets the padding-top value (5px), the second value sets the padding-left and padding-right values (10px), and the third value sets the padding-bottom value (20px).

2 Values

```
p.content-header {  
  
    padding: 5px 10px;  
  
}
```

And finally, if the top and bottom sides can be equal, and the left and right sides can be equal, you can specify 2 values. The first value sets the padding-top and padding-bottom values (5px), and the second value sets the padding-left and padding-right values (10px).

Instructions

1.Change the individual padding declarations on the `h2` selector ruleset to use padding shorthand with 2 values. Keep the padding for the top and bottom at 20 pixels and the value for the left and right padding at 30 pixels.

```
h2 {  
  
    border: 1px dotted red;  
  
    color: red;  
  
    font-size: 14px;  
  
    line-height: 48px;  
  
    text-align: center;  
  
    padding: 20px 30px;  
  
}
```

2.Using two values for the `padding` property, set the padding of the `p` element to 10 pixels on the top and bottom and 20 pixels on the left and right.

```
p {  
  
    color: grey;  
  
    font-size: 16px;  
  
    line-height: 48px;  
  
    padding: 10px 20px;  
  
}
```

Margin

So far you've learned about the following components of the box model: content, borders, and padding. The fourth and final component of the box model is *margin*.

Margin refers to the space directly outside of the box. The `margin` property is used to specify the size of this space.

```
p {  
  
  border: 1px solid aquamarine;  
  
  margin: 20px;  
  
}
```

[Copy to Clipboard](#)

The code in the example above will place 20 pixels of space on the outside of the paragraph's box on all four sides. This means that other HTML elements on the page cannot come within 20 pixels of the paragraph's border.

If you want to be even more specific about the amount of margin on each side of a box, you can use the following properties:

`margin-top`

`margin-right`

`margin-bottom`

`margin-left`

Each property affects the margin on only one side of the box, providing more flexibility in customization.

```
p {
```

```
border: 3px solid DarkSlateGrey;

margin-right: 15px;

}
```

In the example above, only the right side of the paragraph's box will have a margin of 15 pixels. It's common to see margin values used for a specific side of an element.

Instructions

1. Set the top margin of `p` elements to 60 pixels.

```
p {

  color: grey;

  font-size: 16px;

  line-height: 48px;

  padding: 10px 20px;

  margin-top: 60px;

}
```

2. Look at the three red boxes at the bottom of the web page. These elements are anchor elements of class `.share`. Set these `.share` elements to have a margin of 10 pixels.

```
.share {

  border: 1px solid LightGrey;

  padding: 40px 0px;
```

```
position: relative;

text-align: center;

width: 100%;

margin:10px

}

.share a {

background: red;

border: 1px solid red;

border-radius: 3px;

color: white;

display: inline-block;

padding: 14px;

text-decoration: none;

margin:10px;

}
```

Margin Shorthand

What if you don't want equal margins on all four sides of the box and don't have time to separate properties for each side? You're in luck! Margin can be written as a shorthand property as well. The shorthand syntax for margins is the same as padding, so if you're

feeling comfortable with that, skip to the instructions. Otherwise, read on to learn how to use margin shorthand!

Similar to padding shorthand, *margin shorthand* lets you specify all of the `margin` properties as values on a single line:

```
margin-top
```

```
margin-right
```

```
margin-bottom
```

```
margin-left
```

You can specify these properties in a few different ways:

4 Values

```
p {  
  
  margin: 6px 10px 5px 12px;  
  
}
```

In the example above, the four values `6px 10px 5px 12px` correspond to the thickness of the margin on each side, in a clockwise rotation. In order, it specifies the margin-top value (`6px`), the margin-right value (`10px`), the margin-bottom value (`5px`), and the margin-left value (`12px`) of the content.

3 Values

```
p {  
  
  margin: 5px 12px 4px;  
  
}
```

If the left and right sides of the content can be equal, the margin shorthand property allows for 3 values to be specified. The first value sets the margin-top value (`5px`), the

second value sets the margin-left and margin-right values (12px), and the third value sets the margin-bottom value (4px).

2 Values

```
p {  
  
    margin: 20px 10px;  
  
}
```

And finally, if the top and bottom sides can be equal, and the left and right sides can be equal, you can specify 2 values. The first value sets the margin-top and margin-bottom values (20px), and the second value sets the margin-left and margin-right values (10px).

Instructions

1. Using two values, set the h2 top and bottom margins to 30 pixels and the left and right margins to 20 pixels.

```
h2 {  
  
    border: 1px dotted red;  
  
    color: red;  
  
    font-size: 14px;  
  
    line-height: 48px;  
  
    padding: 20px 30px;  
  
    text-align: center;  
  
    margin: 30px 20px;  
  
}
```

Auto

The `margin` property also lets you center content. However, you must follow a few syntax requirements. Take a look at the following example:

```
div.headline {  
  
    width: 400px;  
  
    margin: 0 auto;  
  
}
```

In the example above, `margin: 0 auto;` will center the divs in their containing elements. The 0 sets the top and bottom margins to 0 pixels. The `auto` value instructs the browser to adjust the left and right margins until the element is centered within its containing element.

In order to center an element, a width must be set for that element. Otherwise, the width of the div will be automatically set to the full width of its containing element, like the `<body>`, for example. It's not possible to center an element that takes up the full width of the page, since the width of the page can change due to display and/or browser window size.

In the example above, the width of the `div` is set to 400 pixels, which is less than the width of most screens. This will cause the div to center within a containing element that is greater than 400 pixels wide.

Instructions

1. Set the width of the `.pull-quote` class elements to 350 pixels.

2. In one line, set the vertical (top and bottom) margins of the `.pull-quote` class to `0` and the horizontal (left and right) margins to `auto`.

```
.pull-quote {  
  
    width: 350px;  
  
    margin: 0 auto;  
  
}
```

3. Set the vertical margins of the `#main` element to `0`, and the horizontal margins to `auto`.

```
#main {  
  
    padding: 40px;  
  
    text-align: center;  
  
    width: 400px;  
  
    margin: 0 auto;  
  
}
```

Margin Collapse

As you have seen, padding is space added inside an element's border, while margin is space added outside an element's border. One additional difference is that top and bottom margins, also called vertical margins, *collapse*, while top and bottom padding does not.

Horizontal margins (left and right), like padding, are always displayed and added together. For example, if two divs with ids `#div-one` and `#div-two`, are next to each other, they will be as far apart as the sum of their adjacent margins.

```
#img-one {  
  
    margin-right: 20px;  
  
}  
  
#img-two {  
  
    margin-left: 20px;  
  
}
```

In this example, the space between the `#img-one` and `#img-two` borders is 40 pixels. The right margin of `#img-one` (20px) and the left margin of `#img-two` (20px) add to make a total margin of 40 pixels.

Unlike horizontal margins, vertical margins do not add. Instead, the larger of the two vertical margins sets the distance between adjacent elements.

```
#img-one {  
  
    margin-bottom: 30px;  
  
}  
  
#img-two {  
  
    margin-top: 20px;  
  
}
```

In this example, the vertical margin between the `#img-one` and `#img-two` elements is 30 pixels. Although the sum of the margins is 50 pixels, the margin collapses so the spacing is only dependent on the `#img-one` bottom margin.

It may be helpful to think of collapsing vertical margins as a short person trying to push a taller person. The tall person has longer arms and can easily push the short person, while the person with short arms cannot reach the person with long arms.

Instructions

Study the graphic display to the right. Elements A and B have 20 pixels of horizontal margin, the sum of each element's margin. Elements A and C have 30 pixels of vertical margin — the top margin of element C.



Minimum and Maximum Height and Width

Because a web page can be viewed through displays of differing screen size, the content on the web page can suffer from those changes in size. To avoid this problem, CSS offers two properties that can limit how narrow or how wide an element's box can be sized to:

`min-width`—this property ensures a minimum width of an element's box.

`max-width`—this property ensures a maximum width of an element's box.

```
p {
```

```
min-width: 300px;

max-width: 600px;

}
```

In the example above, the width of all paragraphs will not shrink below 300 pixels, nor will the width exceed 600 pixels.

Content, like text, can become difficult to read when a browser window is narrowed or expanded. These two properties ensure that content is legible by limiting the minimum and maximum widths of an element.

You can also limit the minimum and maximum *height* of an element:

`min-height` — this property ensures a minimum height for an element's box.

`max-height` — this property ensures a maximum height of an element's box.

```
p {

  min-height: 150px;

  max-height: 300px;

}
```

In the example above, the height of all paragraphs will not shrink below 150 pixels and the height will not exceed 300 pixels.

What will happen to the contents of an element's box if the `max-height` property is set too low? It's possible for the content to spill outside of the box, resulting in content that is not legible. You'll learn how to work around this issue in the next exercise.

Instructions

1. In **style.css**, set the minimum width of the `p` element to 200 pixels. After you've done this successfully, resize the browser and notice how the paragraph's box will no longer shrink below 200 pixels.

2. Next, set the maximum width of the `p` element to 800 pixels. After you've done this successfully, resize the browser and notice how the paragraph's box will no longer expand beyond 800 pixels.

3. In **style.css**, set the minimum height of the `p` element to 200 pixels. After you've done this successfully, resize the browser and notice how the height of paragraph's box will no longer shrink below 200 pixels.

4. In **style.css**, set the maximum height of the `p` element to 300 pixels. After you've done this successfully, resize the browser and notice how the height of paragraph's box will no longer expand beyond 300 pixels. You should see your text overflowing. In the next exercise, we will fix that!

```
p {  
  
    color: grey;  
  
    font-size: 16px;  
  
    line-height: 48px;  
  
    margin-top: 60px;  
  
    padding: 10px 20px;  
  
    min-width: 200px;  
  
    max-width: 800px;  
  
    min-height: 200px;  
  
    max-height: 300px;  
}
```



```
}
```

Overflow

All of the components of the box model comprise an element's size. For example, an image that has the following dimensions is 364 pixels wide and 244 pixels tall.

300 pixels wide

200 pixels tall

10 pixels padding on the left and right

10 pixels padding on the top and bottom

2 pixels border on the left and right

2 pixels border on the top and bottom

20 pixels margin on the left and right

10 pixels margin on the top and bottom

The total dimensions (364px by 244px) are calculated by adding all of the vertical dimensions together and all of the horizontal dimensions together. Sometimes, these components result in an element that is larger than the parent's containing area.

How can we ensure that we can view all of an element that is larger than its parent's containing area?

The `overflow` property controls what happens to content that spills, or overflows, outside its box. The most commonly used values are:

`hidden`—when set to this value, any content that overflows will be hidden from view.

`scroll`—when set to this value, a scrollbar will be added to the element's box so that the rest of the content can be viewed by scrolling.

`visible`—when set to this value, the overflow content will be displayed outside of the containing element. Note, this is the default value.

```
p {  
  
  overflow: scroll;  
  
}
```

In the example above, if any of the paragraph content overflows (perhaps a user resizes their browser window), a scrollbar will appear so that users can view the rest of the content.

The overflow property is set on a parent element to instruct a web browser on how to render child elements. For example, if a div's overflow property is set to `scroll`, all children of this div will display overflowing content with a scroll bar.

For a more in-depth look at `overflow`, including additional properties like `overflow-x` and `overflow-y` that separate out the horizontal and vertical values, head over to the [MDN documentation](#).

Instructions

1. In order to see the impact of `overflow: scroll`, first change the height of the `#main` element to 1000 pixels.

2. Set the overflow of the `#main` element to scroll. When you scroll down, a second scroll bar should appear over the paragraph section. You may have to expand the browser component in order to see this behavior clearly.

```
#main {  
  
  margin: 0 auto;
```

```
padding: 40px;

text-align: center;

width: 400px;

height: 1000px;

overflow: scroll;

}
```

Resetting Defaults

All major web browsers have a default stylesheet they use in the absence of an external stylesheet. These default stylesheets are known as *user agent stylesheets*. In this case, the term [user agent](#) is a technical term for the browser.

User agent stylesheets often have default CSS rules that set default values for padding and margin. This affects how the browser displays HTML elements, which can make it difficult for a developer to design or style a web page.

Many developers choose to reset these default values so that they can truly work with a clean slate.

```
* {

margin: 0;

padding: 0;

}
```

The code in the example above resets the default margin and padding values of all HTML elements. It is often the first CSS rule in an external stylesheet.

Note that both properties are set to `0`. When these properties are set to `0`, they do not require a unit of measurement.

Instructions

1. In **style.css**, reset the default margin and padding values for the body. What happens to the web page in the browser?

```
body {  
  
    margin:0;  
  
    padding:0;  
  
    background-color: white;  
  
    font-family: 'Raleway', sans-serif;  
  
}
```

Visibility

Elements can be hidden from view with the `visibility` property.

The `visibility` property can be set to one of the following values:

`hidden` — hides an element.

`visible` — displays an element.

`collapse` — collapses an element.

```
<ul>  
  
    <li>Explore</li>  
  
    <li>Connect</li>
```

```
<li class="future">Donate</li>

</ul>

.future {

  visibility: hidden;

}
```

In the example above, the list item with a class of `future` will be hidden from view in the browser.

Keep in mind, however, that users can still view the contents of the list item (e.g., `Donate`) by viewing the source code in their browser. Furthermore, the web page will *only* hide the contents of the element. It will still leave an empty space where the element is intended to display.

Note: What's the difference between `display: none` and `visibility: hidden`? An element with `display: none` will be completely removed from the web page. An element with `visibility: hidden`, however, will not be visible on the web page, but the space reserved for it will.

Instructions

1. Take a look at the list items in **index.html**. Notice that the list item `Donate` has a class of `donate`. In **style.css**: Add a class selector ruleset for `donate`. Set the visibility to `hidden`.

```
.donate{

  visibility: hidden;

}
```

Why Change the Box Model?

The last lesson focused on the most important aspects of the box model: box dimensions, borders, padding, and margin.

The box model, however, has an awkward limitation regarding box dimensions. This limitation is best illustrated with an example.

```
<h1>Hello World</h1>

h1 {

  border: 1px solid black;

  height: 200px;

  width: 300px;

  padding: 10px;

}
```

In the example above, a heading element's box has solid, black, 1 pixel thick borders.

The height of the box is 200 pixels, while the width of the box is 300 pixels. A padding of 10 pixels has also been set on all four sides of the box's content.

Unfortunately, under the current box model, the border thickness and the padding will affect the dimensions of the box.

The 10 pixels of padding increases the height of the box to 220 pixels and the width to 320 pixels. Next, the 1-pixel thick border increases the height to 222 pixels and the width to 322 pixels.

Under this box model, the border thickness and padding are added to the overall dimensions of the box. This makes it difficult to accurately size a box. Over time, this can also make all of a web page's content difficult to position and manage.

In this brief lesson, you'll learn how to use a different technique that avoids this problem altogether.

Instructions

We'll be using the app provided to demonstrate how to work with the box model. Click "Next" to continue.

Box Model: Content-Box

Many properties in CSS have a default value and don't have to be explicitly set in the stylesheet.

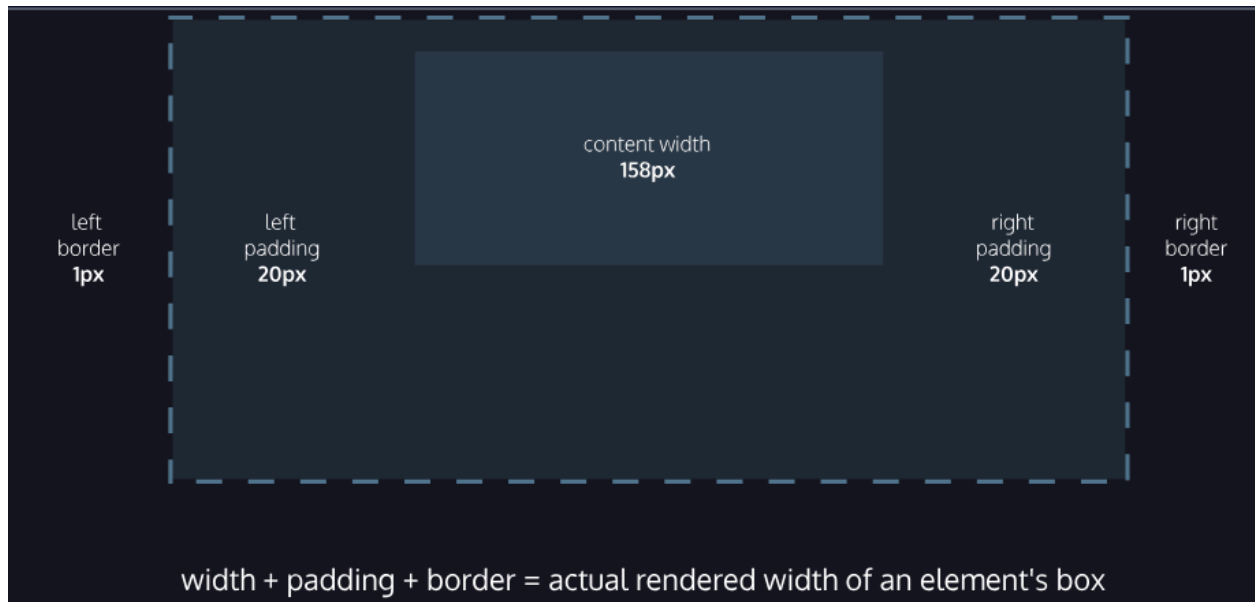
For example, the default `font-weight` of text is `normal`, but this property-value pair is not typically specified in a stylesheet.

The same can be said about the box model that browsers assume. In CSS, the `box-sizing` property controls the type of box model the browser should use when interpreting a web page.

The default value of this property is `content-box`. This is the same box model that is affected by border thickness and padding.

Instructions

Study the diagram to the right. It illustrates the default box model used by the browser, `content-box`. When you're done, continue to the next exercise.



Box Model: Border-Box

Fortunately, we can reset the entire box model and specify a new one: `border-box`.

```
* {  
  
  box-sizing: border-box;  
  
}
```

Copy to Clipboard

The code in the example above resets the box model to `border-box` for all HTML elements. This new box model avoids the dimensional issues that exist in the former box model you learned about.

In this box model, the height and width of the box will remain fixed. The border thickness and padding will be included inside of the box, which means the overall dimensions of the box do not change.

```
<h1>Hello World</h1>
```

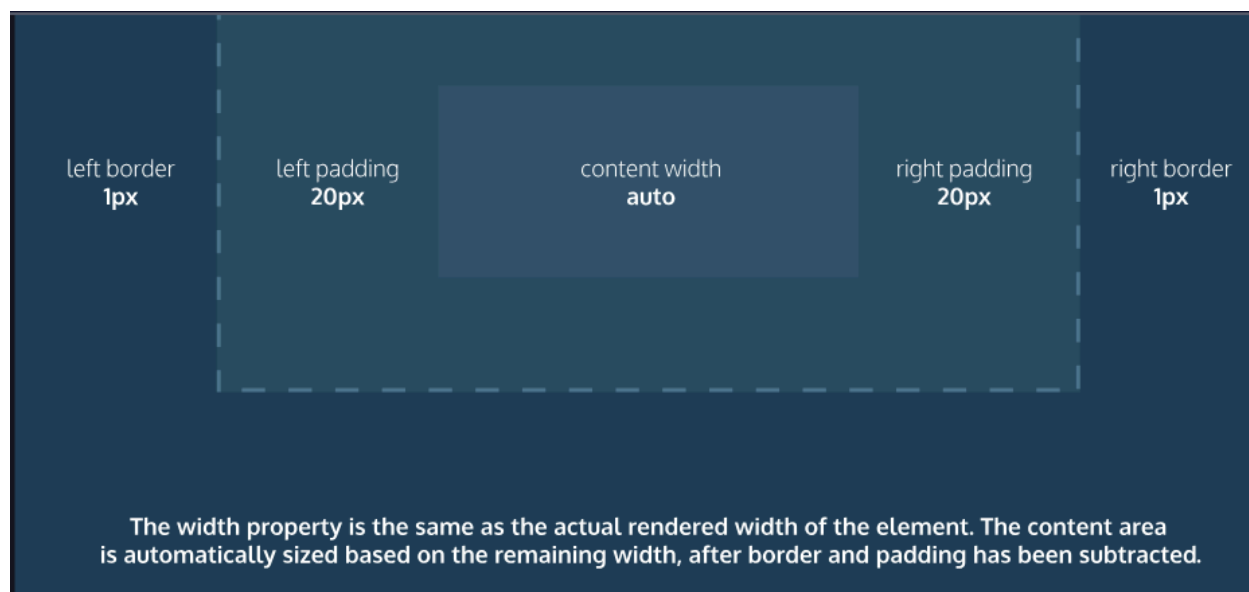


```
* {  
  
  box-sizing: border-box;  
  
}  
  
h1 {  
  
  border: 1px dashed #4f768e;  
  
  height: 150px;  
  
  width: 200px;  
  
  padding: 20px;  
  
}
```

In the example above, the height of the box would remain at 150 pixels and the width would remain at 200 pixels. The border thickness and padding would remain entirely *inside* of the box.

Instructions

Study the diagram to the right. It illustrates the `border-box` model from the code above. Pay attention to how the total width (`200px`) and the padding affect the content width of the element.



The New Box Model

Now that you know about the new box model, let's actually implement it in the browser.

```
* {  
  
  box-sizing: border-box;  
  
}
```

It's that simple! In the example above, the universal selector (*) targets all elements on the web page and sets their box model to the `border-box` model.

Instructions

1. In **style.css**, change the box model for all elements on the web page to the new box model. You probably didn't see a difference in the web page to the right - that's ok! The new box model simply makes sure that the dimensions of elements remains the same regardless of border width and padding.

```
* {  
  
  box-sizing: border-box;  
  
}
```

```
}
```

CSS Display and Positioning

Flow of HTML

A browser will render the elements of an HTML document that has no CSS from left to right, top to bottom, in the same order as they exist in the document. This is called the *flow* of elements in HTML.

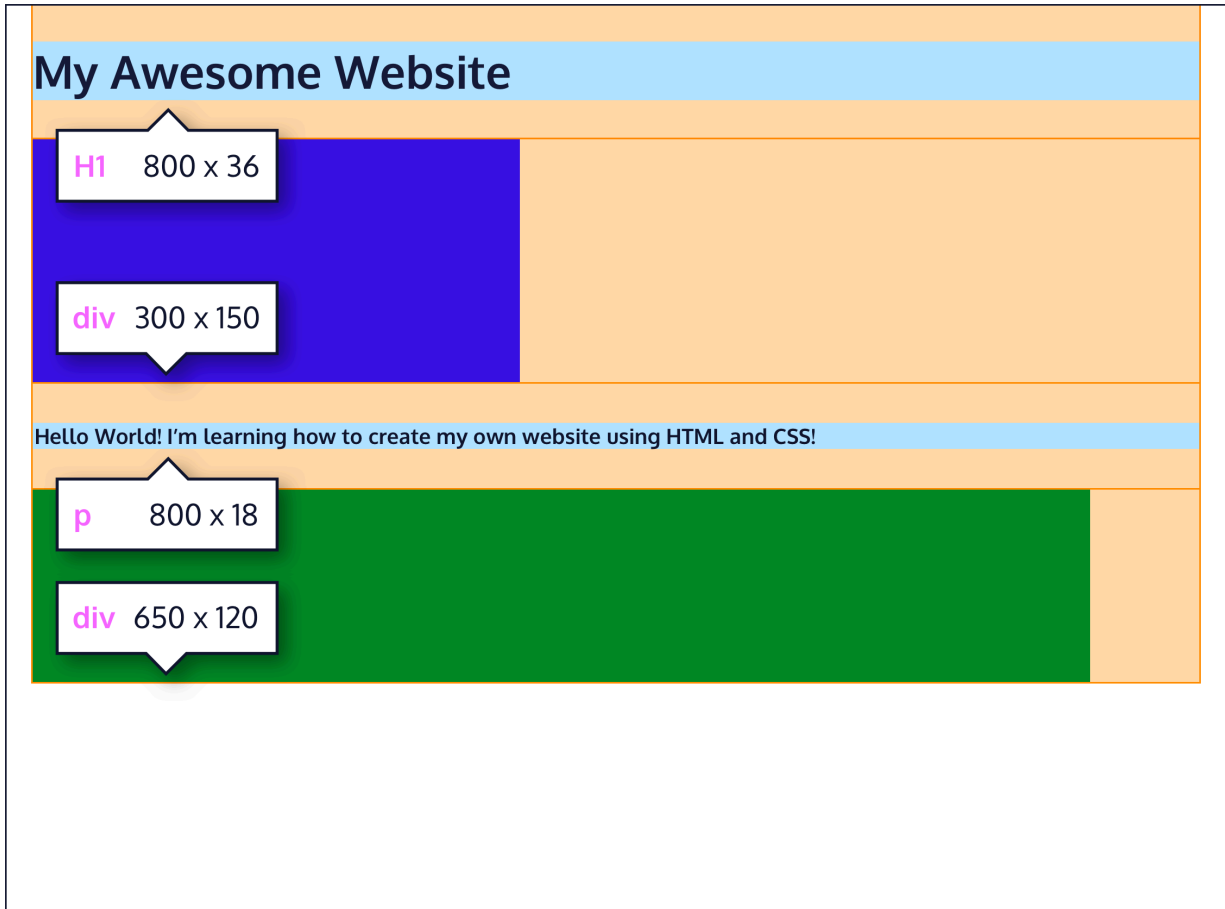
In addition to the properties that it provides to style HTML elements, CSS includes properties that change how a browser *positions* elements. These properties specify where an element is located on a page, if the element can share lines with other elements, and other related attributes.

In this lesson, you will learn five properties for adjusting the position of HTML elements in the browser: `position, display, z-index, float, clear`

Each of these properties will allow us to position and view elements on a web page. They can be used in conjunction with any other styling properties you may know.

Position

Take a look at the *block-level* elements in the image below:



Block-level elements like these boxes create a *block* the full width of their parent elements, and they prevent other elements from appearing in the same horizontal space.

Notice the block-level elements in the image above take up their own line of space and therefore don't overlap each other. In the browser to the right, you can see block-level elements also consistently appear on the left side of the browser. This is the default *position* for block-level elements.

The default position of an element can be changed by setting its `position` property. The `position` property can take one of five values:

`static` - the default value (it does not need to be specified)

`Relative` `absolute` `fixed` `sticky`

In the next few exercises, you'll learn about the values in the list above. For now, it's important to understand that if you favor the default position of an HTML element, you don't need to set its `position` property.

Instructions

1. In **style.css**, add a declaration to the `.question` ruleset that sets the position to static. Notice that setting `position` to `static` does nothing. That's because `static` simply refers to the default behavior.

```
.question {  
  
    text-align: center;  
  
    position: static;  
  
}
```

Position: Relative

One way to modify the default position of an element is by setting its `position` property to `relative`.

This value allows you to position an element *relative* to its default static position on the web page.

```
.green-box {  
  
    background-color: green;  
  
}
```

```
position: relative;

}
```

Although the code in the example above instructs the browser to expect a relative positioning of the `.green-box` element, it does not specify where the `.green-box` element should be positioned on the page. This is done by accompanying the `position` declaration with one or more of the following *offset properties* that will move the element away from its default static position:

`top` - moves the element down from the top.

`bottom` - moves the element up from the bottom.

`left` - moves the element away from the left side (to the right).

`right` - moves the element away from the right side (to the left).

You can specify values in pixels, ems, or percentages, among others, to dial in exactly how far you need the element to move. It's also important to note that offset properties will not work if the element's `position` property is the default `static`.

```
.green-box {

  background-color: green;

  position: relative;

  top: 50px;

  left: 120px;

}
```

In the example above, the element of `green-box` class will be moved down 50 pixels, and to the right 120 pixels, from its default static position. The image below displays the new position of the box.



Offsetting the relative element will not affect the positioning of other elements.

Instructions

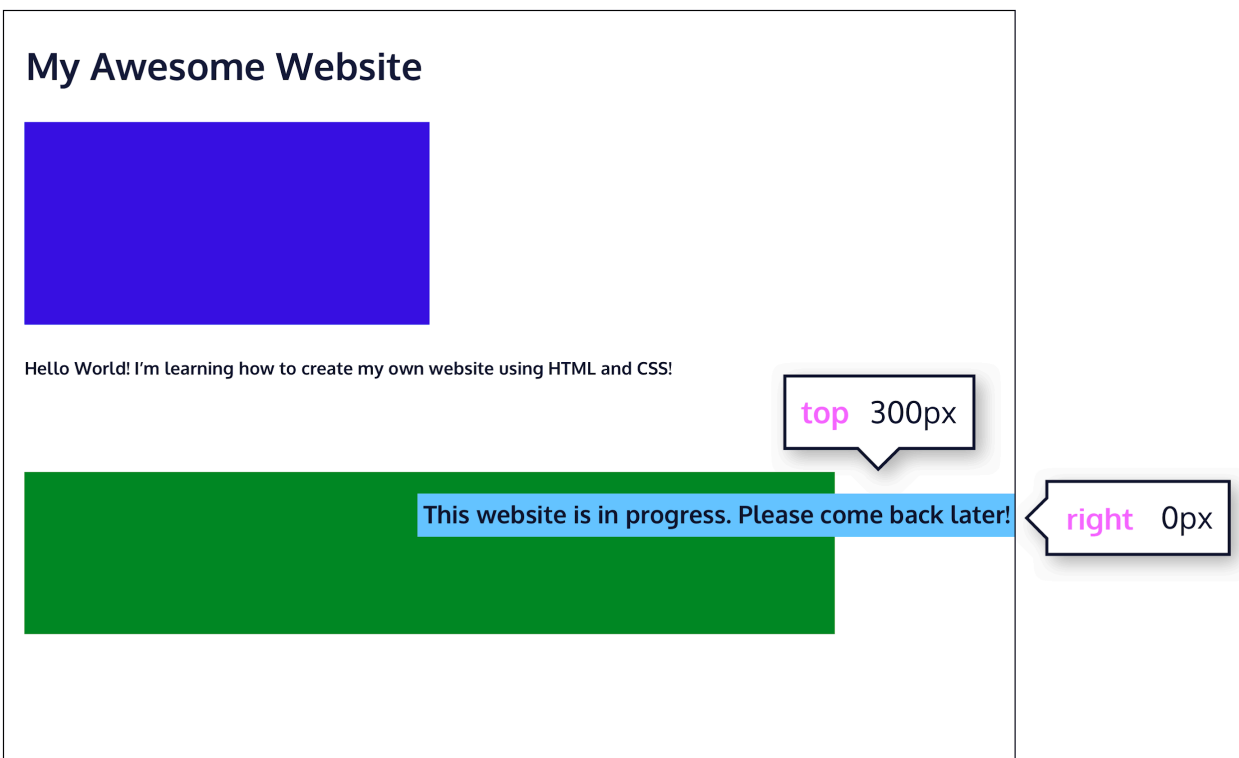
1. In **style.css**, inside the `.question` ruleset, change the `position` property to `relative`.
2. Next, in **style.css**, offset `.question` 40 pixels from the top.

```
.question {  
  
    text-align: center;  
  
    position: relative;  
  
    top: 40px;  
  
}
```

Position: Absolute

Another way of modifying the position of an element is by setting its position to `absolute`.

When an element's position is set to `absolute`, all other elements on the page will ignore the element and act like it is not present on the page. The element will be positioned relative to its closest ancestor that has its position property set to anything other than `static` (e.g., `relative`, `absolute`, or `fixed`), and offset properties can be used to adjust its final position from that reference point. Take a look at the image below:



The "This website is in progress. Please come back later!" is displaced from its static position at the top left corner of its parent container, which is the main white area in the image. It is positioned using absolute positioning, with offset properties of `top: 300px;` and `right: 0px;` moving it 300 pixels down and aligning it with the right edge of the parent

container. This absolute positioning causes other elements on the page to ignore it as if it is not part of the regular document flow.

Instructions

1. In **style.css**, set the `position` inside of the `header` ruleset to `absolute`. Notice how it's now removed from the normal flow of the document, and covering the welcome section.

2. When you changed the position to absolute, you may have noticed that the header shrunk horizontally. We'll learn why in a later exercise. For now, set the `width` property of the header to `100%`.

```
header {  
  
    background-color: #466995;  
  
    border-bottom: 1px solid #466995;  
  
    position: absolute;  
  
    width: 100%;  
  
}
```

Position: Fixed

When an element's position is set to `absolute`, as in the last exercise, the element will scroll with the rest of the document when a user scrolls.

We can *fix* an element to a specific position on the page (regardless of user scrolling) by setting its position to `fixed`, and accompanying it with the familiar offset properties `top`, `bottom`, `left`, and `right`.

```
.title {
```

```
position: fixed;

top: 0px;

left: 0px;

}
```

In the example above, the `.title` element will remain fixed to its position no matter where the user scrolls on the page, like in the image below:

This technique is often used for navigation bars on a web page.

Instructions

1. In **style.css**, change the `position` property inside of the `header` rule to `fixed`. Scroll up and down the web page. What do you notice?

2. Notice that part of the “Welcome” section is now covered up by the header. That’s because when we changed the position of the header to fixed, we removed it from the flow of the html document. Let’s fix that. Set the `position` of the `.welcome` element to `relative`.

3. Offset the “Welcome” section by 200 pixels from the top. Everything might not be displaying correctly just yet; we’ll fix it in a later exercise.

```
header {

    background-color: #466995;

    border-bottom: 1px solid #466995;

    position: fixed;

    width: 100%;

}
```

```
.welcome {  
  
  background-color: #DBE9EE;  
  
  box-sizing: border-box;  
  
  padding: 40px;  
  
  text-align: center;  
  
  width: 100%;  
  
  position: relative;  
  
  top: 200px;  
  
}
```

Position: Sticky

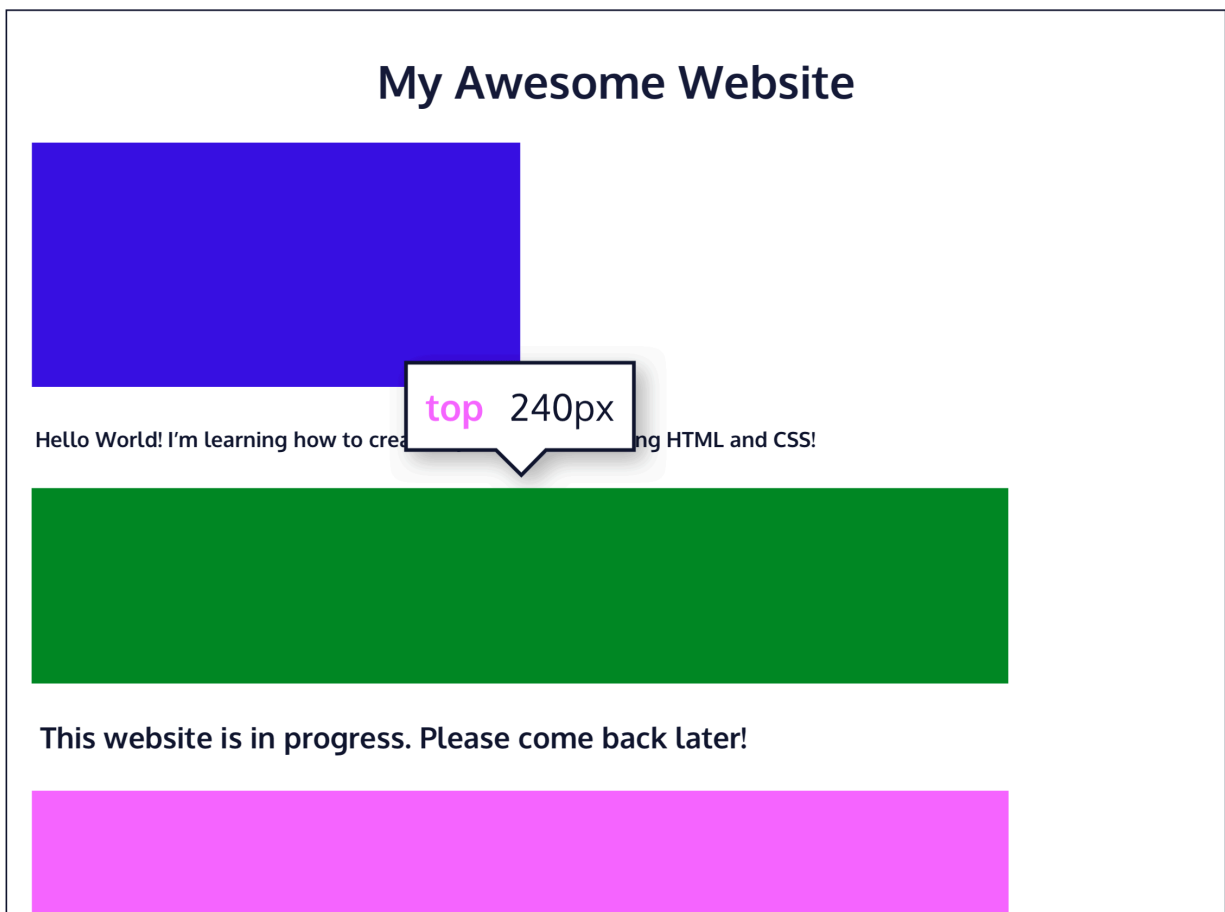
Since `static` and `relative` positioned elements stay in the normal flow of the document, when a user scrolls the page (or parent element) these elements will scroll too. And since `fixed` and `absolute` positioned elements are removed from the document flow, when a user scrolls, these elements will stay at their specified offset position.

The `sticky` value is another position value that keeps an element in the document flow as the user scrolls, but *sticks* to a specified position as the page is scrolled further. This is done by using the `sticky` value along with the familiar offset properties, as well as one new one.

```
.box-bottom {  
  
  background-color: darkgreen;  
  
  position: sticky;
```

```
top: 240px;  
  
}
```

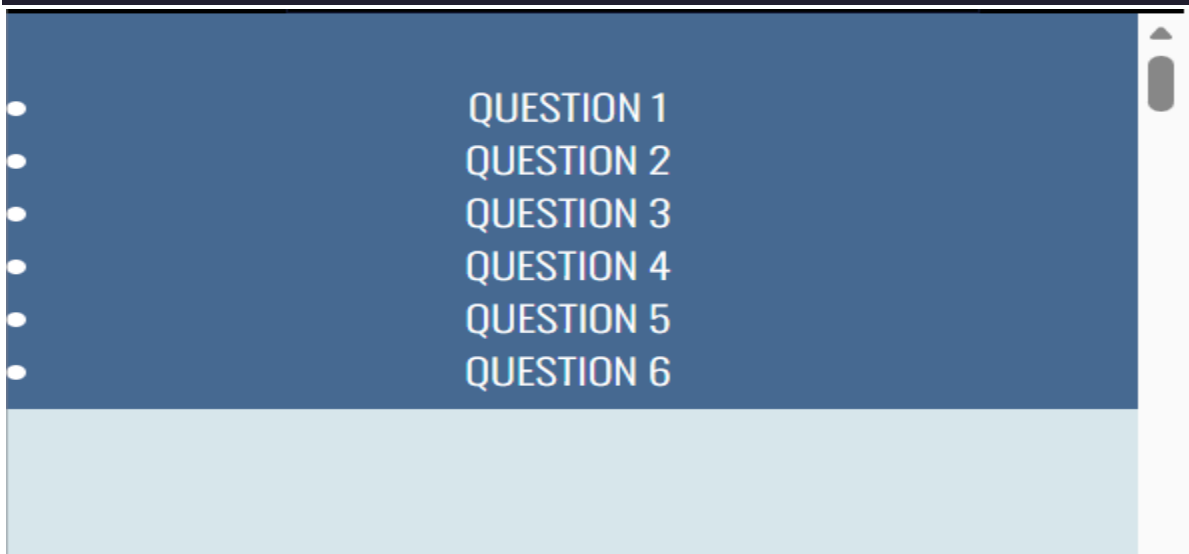
In the example above, the `.box-bottom <div>` will remain in its relative position, and scroll as usual. When it reaches 240 pixels from the top, it will stick to that position until it reaches the bottom of its parent container where it will “unstick” and rejoin the flow of the document.



Instructions

1. Inside **style.css**, change the position of the elements with the class `question` to `sticky`.

```
.question {  
  
    text-align: center;  
  
    position: sticky;  
  
    top: 40px;  
  
    border: 15px solid #DBE9EE;  
  
    background-color: white;  
  
}
```



Z-Index

When boxes on a web page have a combination of different positions, the boxes (and therefore, their content) can overlap with each other, making the content difficult to read or consume.

```
.blue-box {  
  
    background-color: blue;  
  
}  
  
.green-box {  
  
    background-color: green;  
  
    position: relative;  
  
    top: -170px;  
  
    left: 170px;  
  
}
```

Copy to Clipboard

In the example above, the `.green-box` element overlaps on top of the `.blue-box` element.

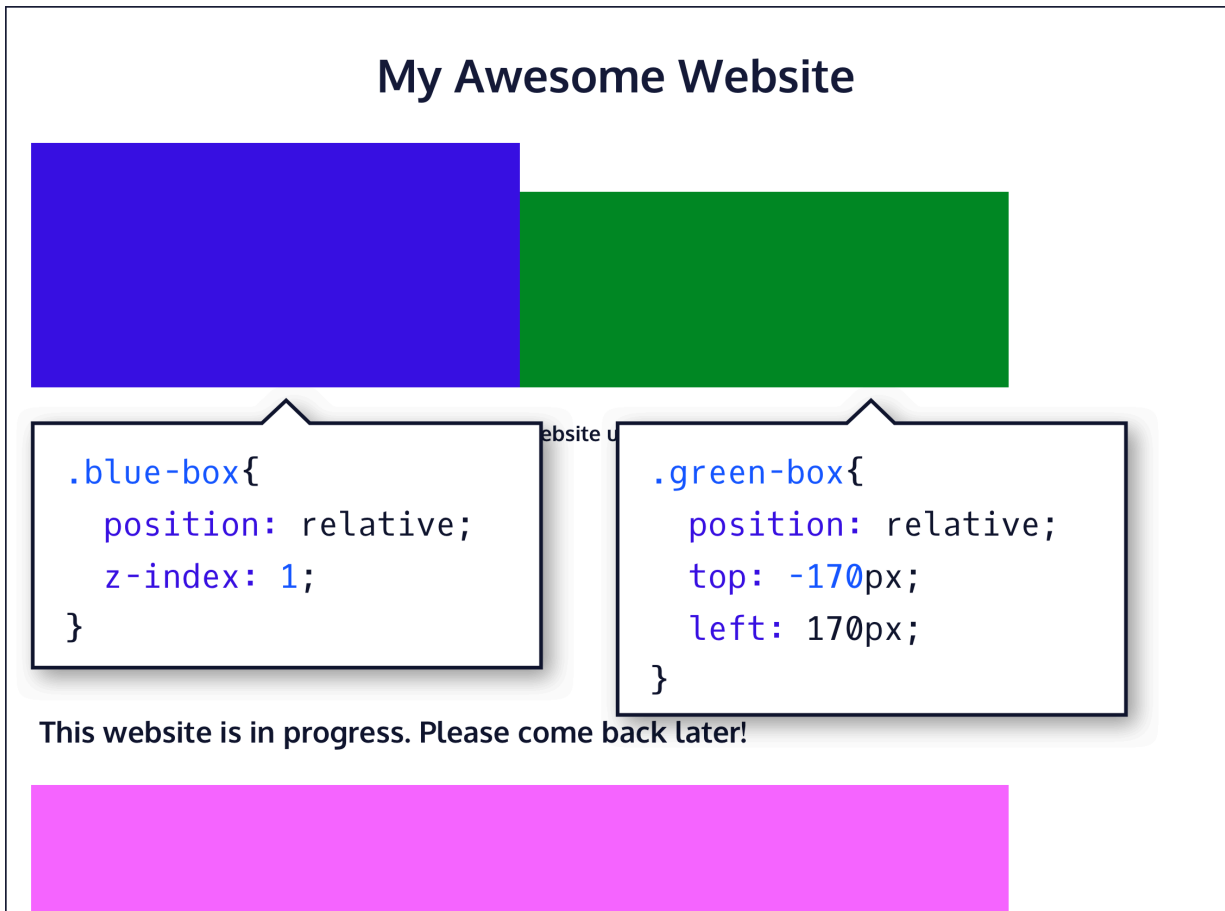
The `z-index` property controls how far back or how far forward an element should appear on the web page when elements overlap. This can be thought of as the *depth* of elements, with deeper elements appearing behind shallower elements.

The `z-index` property accepts integer values. Depending on their values, the integers instruct the browser on the order in which elements should be layered on the web page.

```
.blue-box {  
  
  background-color: blue;  
  
  position: relative;  
  
  z-index: 1;  
}  
  
.green-box {  
  
  background-color: green;  
  
  position: relative;  
  
  top: -170px;  
  
  left: 170px;  
}
```

Copy to Clipboard

In the example above, we set the `.blue-box` position to `relative` and the z-index to 1. We changed position to `relative`, because the `z-index` property does *not* work on static elements. The z-index of 1 moves the `.blue-box` element forward, because the `z-index` value has not been explicitly specified for the `.green-box` element, which means it has a default `z-index` value of 0. Take a look at the example image below:



Instructions

1. In **style.css**, set the `z-index` of the `header` to `10`. Notice how the header is no longer covered by other elements when you scroll!

```
header {

  background-color: #466995;

  border-bottom: 1px solid #466995;

  position: relative;

  width: 100%;

  Z-index:10; }
```




Inline Display

Every HTML element has a default `display` value that dictates if it can share horizontal space with other elements. Some elements fill the entire browser from left to right regardless of the size of their content. Other elements only take up as much horizontal space as their content requires and can be directly next to other elements.

In this lesson, we'll cover three values for the `display` property: `inline`, `block`, and `inline-block`.

The default `display` for some elements, such as `<em>`, `<strong>`, and `<a>`, is called *inline*. Inline elements have a box that wraps tightly around their content, only taking up the amount of space necessary to display their content and not requiring a new line after each element. The height and width of these elements cannot be specified in the CSS document. For example, the text of an anchor tag (`<a>`) will, by default, be displayed on the same line as the surrounding text, and it will only be as wide as necessary to contain its content. `inline` elements cannot be altered in size with the `height` or `width` CSS properties.

```
To learn more about <em>inline</em> elements, read <a href="#">MDN documentation</a>.
```

In the example above, the `<em>` element is `inline`, because it displays its content on the same line as the content surrounding it, including the anchor tag. This example will display:

To learn more about *inline* elements, read [MDN documentation](#).

The CSS `display` property provides the ability to make any element an inline element. This includes elements that are not inline by default such as paragraphs, divs, and headings.

```
h1 {  
  
    display: inline;  
  
}
```

The CSS in the example above will change the display of all `<h1>` elements to `inline`.

The browser will render `<h1>` elements on the same line as other inline elements immediately before or after them (if there are any).

Instructions

1. In `index.html`, add opening and closing `<strong></strong>` tags around the word “Welcome”. Notice that the element does not move. That’s because `<strong>` elements are inline, so they can share lines with other elements.

```
<div class="welcome">  
  
    <h1><strong>Welcome</strong> to our survey!</h1>  
  
    <p>We're looking forward to getting your answers so we can make sure  
our products and services are the best they can be!</p>
```

```
</div>
```

Display: Block

Some elements are not displayed in the same line as the content around them. These are called *block-level* elements. These elements fill the entire width of the page by default, but their `width` property can also be set. Unless otherwise specified, they are the height necessary to accommodate their content.

Elements that are block-level by default include all levels of heading elements (`<h1>` through `<h6>`), `<p>`, `<div>` and `<footer>`. For a complete list of block level elements, visit [the MDN documentation](#).

```
strong {  
  
    display: block;  
  
}
```

In the example above, all `<strong>` elements will be displayed on their own line, with no content directly on either side of them even though their contents may not fill the width of most computer screens.

Instructions

1. In `index.html`, add a `<footer>` element at the bottom of the document, right above the closing `</body>` tag.

2. Nothing changed! That's because the `<footer>` element is empty. Add an `<h3>` element inside of the footer that says "Thanks for taking our survey!"

3. To improve the appearance of the web page, set the height of the footer to 100 pixels in `style.css`.

```
<footer>

  <h3>Thanks for taking our survey!</h3>

</footer>

footer {

  background-color: #DBE9EE;

  text-align: center;

  height:100px;

}
```

Display: Inline-Block

The third value for the `display` property is `inline-block`. Inline-block display combines features of both inline and block elements. Inline-block elements can appear next to each other and we can specify their dimensions using the `width` and `height` properties. Images are the best example of default inline-block elements.

For example, the `<div>`s below will be displayed on the same line and with the specified dimensions:

I'm a rectangle!

So am I!

Me three!

Let's take a look at the code:

```
<div class="rectangle">

  <p>I'm a rectangle!</p>

</div>

<div class="rectangle">

  <p>So am I!</p>

</div>

<div class="rectangle">

  <p>Me three!</p>

</div>

.rectangle {

  display: inline-block;
```

```
width: 200px;

height: 300px;

}
```

There are three rectangular divs that each contain a paragraph of text. The `.rectangle` `<div>`s will all appear inline (provided there is enough space from left to right) with a width of 200 pixels and height of 300 pixels, even though the text inside of them may not require 200 pixels by 300 pixels of space.

Instructions

1. Let's fix the display of the list elements in the menu at the top of the page. Set the `display` property of `<li>` elements to `inline-block`.
2. Set the width of the `li` elements to 80 pixels.
3. Now, we can reduce the top offset of the "Welcome" section. Set it to 50 pixels.
4. Set the `display` property of `.answer` elements to `inline-block`.

```
.welcome {

  background-color: #DBE9EE;

  box-sizing: border-box;

  padding: 40px;

  text-align: center;

  width: 100%;

  position: relative;

  top: 50px;

}
```

```
.answer:hover {  
  
  background: #C0D6DF;  
  
  color: #FFF;  
  
}
```

Float

So far, you've learned how to specify the exact position of an element using offset properties. If you're simply interested in moving an element as far left or as far right as possible in the container, you can use the `float` property.

The `float` property is commonly used for wrapping text around an image. Note, however, that moving elements left or right for layout purposes is better suited for tools like CSS grid and flexbox, which you'll learn about later on.

The `float` property is often set using one of the values below:

`left` - moves, or floats, elements as far left as possible.

`right` - moves elements as far right as possible.

```
.green-section {  
  
  width: 50%;  
  
  height: 150px;  
  
}  
  
.orange-section {  
  
  background-color: orange;
```

```
width: 50%;  
  
float: right;  
  
}
```

In the example above, we float the `.orange-section` element to the `right`. This works for static and relative positioned elements. See the result of the code below:



Floated elements must have a width specified, as in the example above. Otherwise, the element will assume the full width of its containing element, and changing the float value will not yield any visible results.

Instructions

1. Add a declaration to the `.answer` ruleset that sets the `float` property to `left`.


```
.answer {  
  
  border: 1px solid #466995;  
  
  margin: 20px;  
  
  float: left;  
  
}
```

Clear

The `float` property can also be used to float multiple elements at once. However, when multiple floated elements have different heights, it can affect their layout on the page. Specifically, elements can “bump” into each other and not allow other elements to properly move to the left or right.

The `clear` property specifies how elements should behave when they bump into each other on the page. It can take on one of the following values:

`left`—the left side of the element will not touch any other element within the same containing element.

`right`—the right side of the element will not touch any other element within the same containing element.

`both`—neither side of the element will touch any other element within the same containing element.

`none`—the element can touch either side.

```
div {  
  
  width: 200px;
```

```
float: left;

}

div.special {

    clear: left;

}
```

In the example above, all `<div>`s on the page are floated to the left side. The element with class `special` did not move all the way to the left because a taller `<div>` blocked its positioning. By setting its `clear` property to `left`, the `special <div>` will be moved all the way to the left side of the page.

Instructions

1. Take a look at the `.answer` divs on the web page. They have been floated to the left, but the `.question` divs are touching the `.answer` divs on the right, let's fix this. In the `.question` selector, set the `clear` property to `left`. Notice how the questions moved.

2. On second thought, this layout is not looking so good. Remove the `float` property from `.answer` selector ruleset.

```
.question {

    text-align: center;

    position: relative;

    top: 40px;

    clear: left;

}
```

```
.answer {  
  
  border: 1px solid #466995;  
  
  margin: 20px;  
  
  display: inline-block;  
  
}
```